Replicating a Texting Voice

Building and honestly evaluating a fine-tuned persona model of one person, against a production RAG
+ GPT-4o-mini baseline

Bohdan Chuprynka · June 2026

Abstract. Can a small local fine-tune replicate one person's texting voice — and how would you prove it? We build Persona-RAG, a Telegram bot that answers in its owner's voice, and ask whether a fine-tuned Qwen2.5-3B LoRA texts more like him than the shipped gpt-4o-mini product (a ~1600-token retrieval-augmented prompt with decode levers). Trust comes first: we document a ~90% train/test leak in the original evaluation, found and fixed, then score both backends on a recipient-stratified, model-disjoint hold-out with paired bootstrap intervals under a pre-registered acceptance rule. A controlled arm isolates the weights; a production arm pits the full API stack against the thin fine-tune, with a per-item retrieval guard that removes a measured 28% gold-answer leak. On a level field the fine-tune is decisively closer on reply length (Cliff's $\delta = 0.95$, closer on 292 of 299 messages) and matches the no-"!" register; fully equipped, the production machinery only pulls the API back to a *voice tie* — at $0.37 per thousand replies and an ~11× token tax against $0 local — which cost, privacy, and offline capability break toward the local model. On the dimensions we can measure, then, the conclusion is clear: the local fine-tune is at least the equal of the fully-equipped product and decisively beats the bare model — the better replica of the register, at zero cost. What these surface metrics cannot certify is the subjective "feels like me"; that pre-registered *primary* check (a blind human win-rate) is unresolved here, because the only rater with standing (the author) is recall-biased and privacy precludes outside raters — a real limitation, not a gap in the metric verdict.

1 Building the replica

This part is the implementation story: what it takes to make a model that texts like one specific person, and the design choices that the evaluation later has to account for. The running subject is a fine-tuned *Qwen2.5-3B* LoRA [1], [2] served locally, set against the shipped OpenAI gpt-4o-mini product it was built to replace.

1.1 The target: what "sounds like me" means

The system has exactly one thing it must get right — *does this read like something the owner would actually text?* That target is subjective but it has measurable fingerprints: heavy Ukrainian/Russian/English code-switching (about 0.18 of characters in Latin script, aggregated across contacts); terse, multi-bubble bursts (several short messages rather than one paragraph); a `)` smiley tic; almost no `!`; varied openers; lowercase casing. The deployed gpt-4o-mini was capable and fluent, but on these register markers it did not sound like the person — which is what motivated a fine-tune.

1.2 System architecture

Each inbound Telegram message runs through a single LangGraph pipeline (Figure 1): authentication, hybrid retrieval, contact memory, self-insights, prompt assembly, generation, guardrails, and bubble-wise delivery. The generation node fans out to two interchangeable backends — the OpenAI API (gpt-4o-mini) and a local `llama.cpp` server hosting the fine-tuned LoRA. A single `GENERATION_BACKEND` switch selects which, and routes the local path to *skip* retrieval entirely: the fine-tune serves on the same thin prompt it trained on.

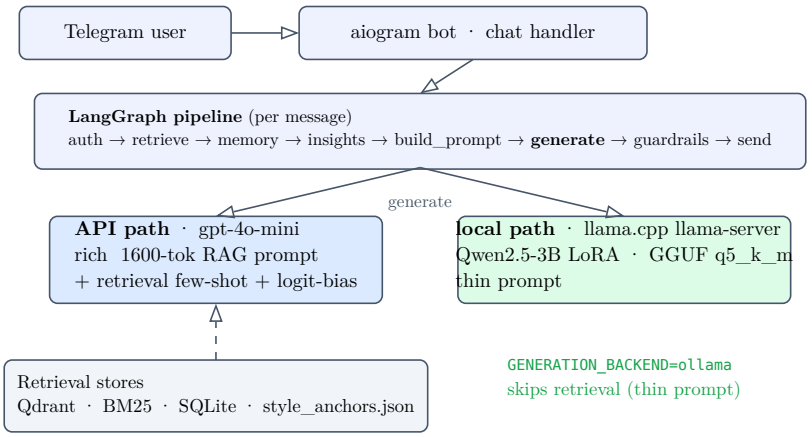

Figure 1: Persona-RAG. One LangGraph pipeline; the generation node fans to two backends. The API path carries a rich retrieval-augmented prompt and decode levers; the local LoRA path skips retrieval and serves a thin prompt.

1.3 Data, turns, and the split problem

Raw Telegram/Instagram exports are PII-redacted, collapsed into bursts (messages within 300s of each other), cut into sessions on 6-hour idle gaps, and reduced to *persona turns*: each turn is one reply the owner sent, paired with the preceding context. This `(context → reply)` pair is the atomic unit for both retrieval and fine-tuning.

Two different held-out splits exist, and the distinction is load-bearing for the evaluation (Figure 2). The product indexes and serves against a *temporal* split (the most recent 10% of turns by timestamp). But the temporal tail is register-skewed: a single English-speaking contact accounts for 62% of all Latin script in the corpus, so the recent slice reads about 0.47 Latin against the person's true ~0.18 — an artifact that makes the voice target unreachable. The fine-tune therefore trains and evaluates on a *recipient-stratified* split (`eval_split_for`), a deterministic hash that holds out ~10% of *every* contact independently, so train and eval share the same code-switch register (train ≈ eval ≈ 0.18). Every fair comparison in this report scores on that recipient-stratified, LoRA-disjoint hold-out.

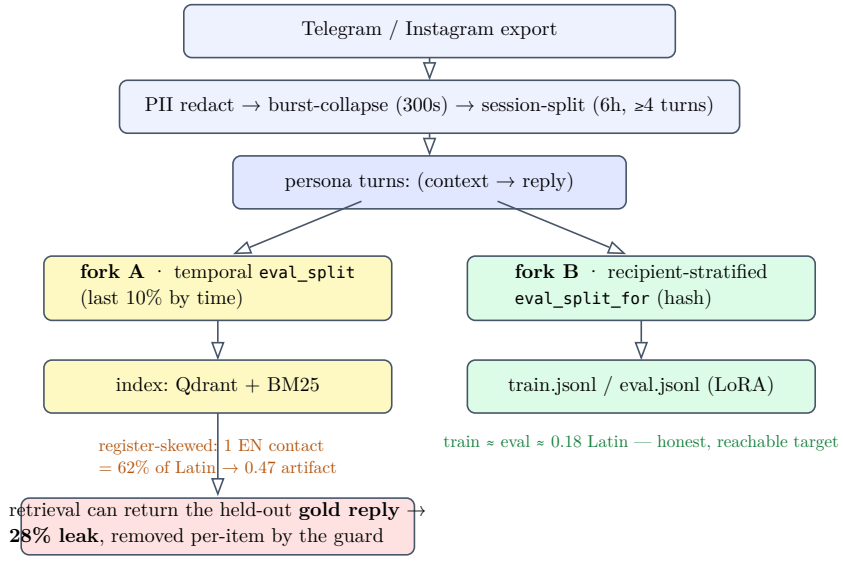

Figure 2: Data pipeline and the two hold-out splits. The temporal split (left) drives indexing but is register-skewed; the recipient-stratified split (right) gives the fine-tune an honest, reachable voice target. The retrieval leak that the temporal index enables is removed per-item (Figure 5).

1.4 Retrieval

The API path retrieves the owner's own past replies as few-shot examples. Dense search (`text-embedding-3-small` over Qdrant) and BM25 [3] are min-max-normalised and fused (0.7 weight on dense), decayed by recency (180-day half-life), floored at a minimum score, and diversified by Maximal Marginal

Relevance [4] ($\lambda = 0.6$) down to the four best, which are then reversed so the single most relevant example lands last. A parallel store of distilled "self-insights" adds a bio-facts block. All of it renders into a ~1600-token system prompt with explicit, enforced voice rules.

1.5 The fine-tune, and the train==serve invariant

The fine-tune is a QLoRA [5] adapter (rank 32, $\alpha = 64$) on Qwen2.5-3B-Instruct, trained with Unsloth [6] on a free Colab T4. Loss is masked to the assistant turn only, and multi-bubble newlines are preserved so the model learns the burst shape. Crucially, the system turn used in training is the byte-identical one-line persona anchor used at serving time (Figure 3) — any drift between the two would be train/serve skew. After training, the adapter is merged and then converted and quantized *locally* to a GGUF `Q5_K_M` model, served through `llama.cpp`'s OpenAI-compatible server [7] (the Homebrew Ollama formula on the target machine shipped no inference runtime).

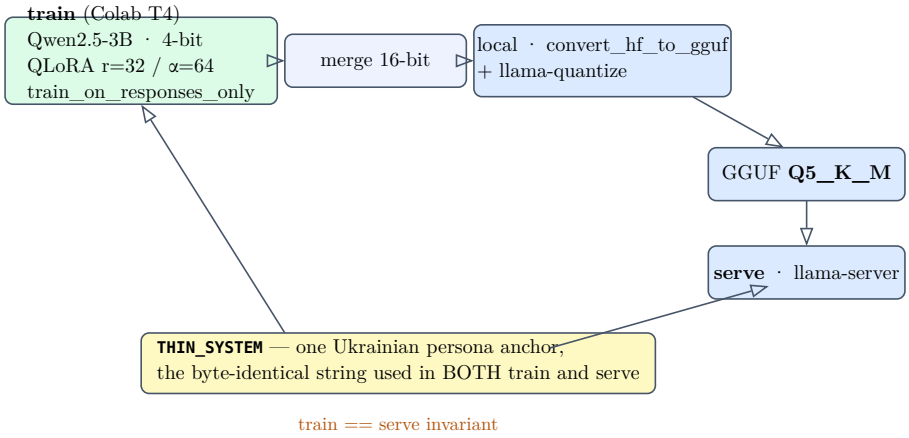

Figure 3: Train==serve. The same thin persona-anchor string conditions both training and serving; the adapter is merged, quantized to GGUF Q5_K_M, and served locally.

Why fine-tune at all, rather than prompt harder? Message *shape* was already solved by prompt shape-conditioning. But lexical voice — the code-switch, opener variety, the `)` tic, the absent `!` — does not respond to soft prompt instructions: the base model obeys hard per-reply directives but ignores style requests. Those tics had to be learned from the data. The production API path compensates differently, with two decode levers (a positive logit bias on the `)` token, a strong negative bias on `!`) and per-reply register directives — machinery the next part puts on trial.

2 Can we trust the measurement?

Before any comparison can mean anything, the evaluation itself has to be trustworthy. This part is the part most write-ups skip: the metrics, the leak that was found and fixed, the harness, and the acceptance rule fixed *before* the results.

2.1 Measuring a voice

No single number captures "sounds like the person", so the evaluation uses a small vector of surface metrics, each targeting one of the voice fingerprints from Part I. The two headline distances compare *distributions* against the real held-out replies:

- **Message shape** (shape_{JS}): the Jensen-Shannon divergence between the generated and real distributions of bubble-count per reply (how many separate messages a reply is split into). Bounded to $[0, 1]$; lower is closer. It catches a constant-3-bubble mode collapse that a mean would score as perfect.
- **Reply length** (W_1): the 1-Wasserstein (earth-mover) distance [8] between the per-bubble character-length distributions, in characters. Lower is closer.

Alongside these run a set of register rates: the exclamation rate, the `)` smiley rate (detected by unbalanced close-parens so real parentheticals do not false-positive), the Latin-script rate (the code-switch signal), opener entropy ($H = -\sum_i p_i \log p_i$ over first words — higher is more varied), and the

distinct-reply rate (a mode-collapse guard). A copy / near-copy rate against the training replies guards against memorization, but only ever read against a measured *natural floor* — the rate at which the person's own held-out replies coincide with their past replies, because short casual texts ("ok", "+") recur naturally.

2.2 The audit: a ~90% leak, found and fixed

The project's original evaluation runner was structurally unfair, on a disqualifying count. It scored the model on the *temporal* `eval_split`, but the fine-tune had held out the *recipient-stratified* `eval_split_for` (Figure 2). The two are nearly disjoint partitions, so roughly **90% of the "held-out" turns the runner scored were in the fine-tune's training pool**, while the API had seen none — a one-sided leak that flattered the fine-tune on every metric. The same runner also confounded the backend with the prompt, the retrieval, and the decode levers (all of which move together with "which backend"), and reported point estimates with no uncertainty at all. No "fine-tune beats RAG" claim from that runner was defensible. Catching this is the foundation everything else rests on.

2.3 The fair harness

The replacement is a small, pure, unit-tested scoring core. Both backends are scored on the *same* recipient-stratified, LoRA-disjoint hold-out. Every headline distance carries a **paired bootstrap 95% confidence interval** (2000 resamples), and a difference counts only if its interval excludes zero. Degenerate generations return `NaN`, never `0.0`, so an all-empty backend can never score an artificial perfect distance. The screen runs at $n = 300$, which drops the shape-divergence sampling-noise floor from ~0.06 (at $n = 80$) to ~0.02.

2.4 Two arms: model versus product

A single comparison cannot answer both "which *model* is better" and "which *product* ships better", because the deployed API bundles a rich prompt, retrieval, and decode levers with its weights. So the evaluation runs two arms (Figure 4). **Arm B (controlled)** gives both backends the identical thin prompt, with no retrieval, directives, or levers — it isolates the weights. **Arm A (production)** pits the fully shipped API stack against the thin LoRA — it measures the real deployed gap.

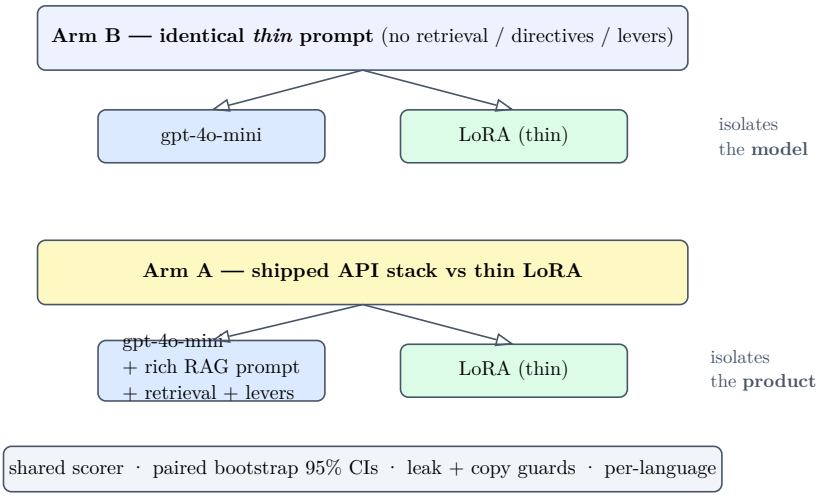

Figure 4: The two-arm design. Arm B isolates the model (identical thin prompt); Arm A isolates the product (shipped API stack vs. thin LoRA). Both feed one scorer with paired bootstrap CIs and the leak/copy guards.

2.5 The retrieval leak guard

The production arm has its own contamination risk: because the held-out gold turns sit in the very corpus the API retrieves from, retrieval can hand the model the exact answer key for the item being scored. Measured directly, with exclusion disabled, **28% of items (17 of 60)** retrieved their own gold reply into the few-shot pool. A per-item guard excludes the scored turn's id from retrieval, driving the **exact answer-key** — the gold turn by id, and its verbatim text under the same context — to

zero, while leaving the mean top-1 similarity essentially unchanged (0.386 vs. 0.389): the guard removes contamination, not retrieval quality (Figure 5). It is an exact-match guard, so a near-paraphrase under a different id could in principle slip through; here no retrieved neighbour exceeded 0.9 similarity, but that residual near-duplicate risk is named among the limitations.

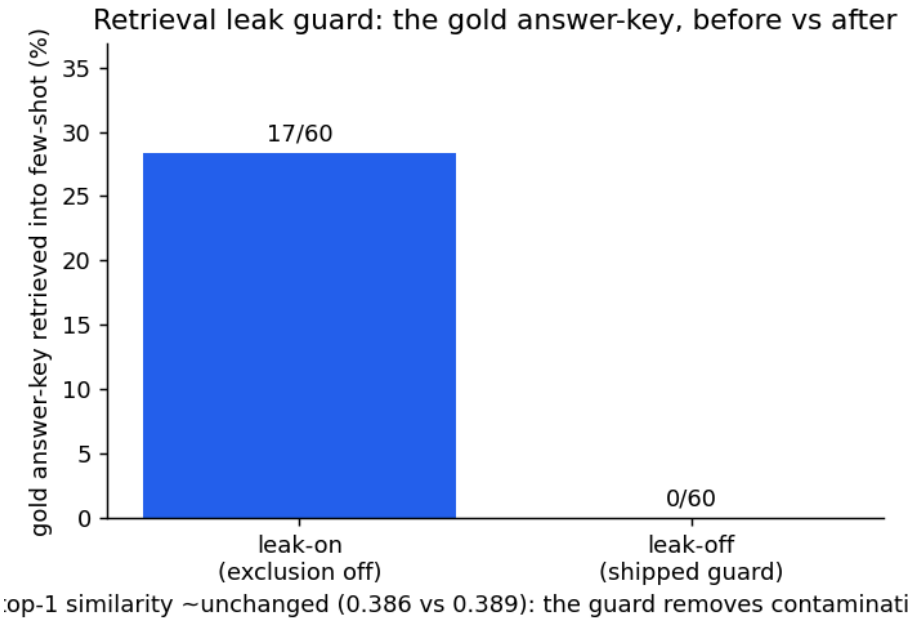

Figure 5: The retrieval leak guard. Disabling exclusion lets the API retrieve the held-out gold reply for 28% of items; the per-item guard removes all of it without degrading top-1 similarity. $n = 60$ per condition.

2.6 The rule, fixed in advance

To defend against metric-shopping, the definition of "better" and the ship rule were fixed before the results. The **primary verdict is the blind human win-rate**: a backend wins on voice only if its Wilson 95% interval over the other excludes 0.5; otherwise it is a voice tie. Guardrails override a numeric win — a win by memorization (copy rate above the natural floor) or mode collapse (distinct-reply rate too low) does not count. And ties break toward the local fine-tune on cost, latency, and offline capability, because those are its reason for existing.

3 Relative fidelity

The first fidelity question is relative: does the fine-tune beat the *strong* baseline — the full gpt-4o-mini product — at sounding like the person? This part answers it across the two arms, with ablations that explain *why*, and an effect size that keeps the answer honest.

3.1 Arm B: on a level playing field

Under the identical thin prompt, the fine-tune is decisively closer on length and matches the no-`!` rule exactly, while message shape is a statistical tie. The result replicates across two seeds (Table 1, Figure 6).

metric (↓ closer)	API	LoRA	Δ (95% CI)	verdict
`shape_js`	0.052	0.024	+0.028 (-0.014, 0.073)	tie
`len_wasserstein`	128.8	**2.9**	+125.9 (107.6, 142.4)	**LoRA**
`exclaim_rate`	0.651	0.000	—	descr.
`opener_entropy` (↑)	5.02	5.77	—	descr.
cost / 1k replies	$0.082	**$0**	—	LoRA

Table 1: Arm B (controlled), $n = 300$, replicated at $n = 150$ seed 1. Identical thin prompt to both backends. The length win is large and its CI excludes zero in both seeds.

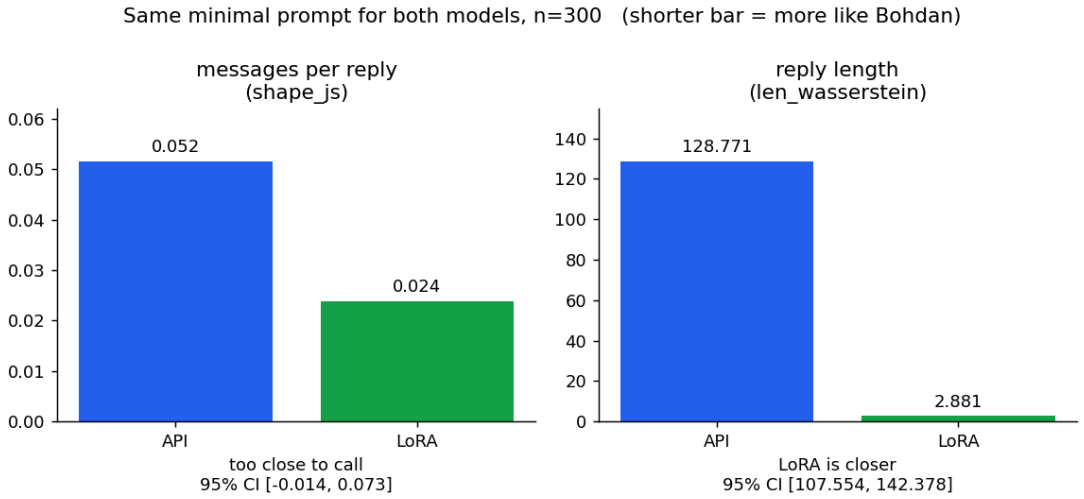

Figure 6: Arm B headline distances. Message shape is a tie; reply length favors the LoRA decisively (shorter is more like the person).

3.2 Arm A: the product fully equipped

Giving the API its entire shipped advantage — rich prompt, retrieval, levers — changes the picture but does not overturn it (Table 2, Figure 7). Shape and `!` are now ties; the LoRA stays ahead on reply-length distribution and opener variety, at zero cost.

metric (↓ closer)	API	LoRA	Δ (95% CI)	verdict
`shape_js`	0.0353	0.0339	+0.001 (-0.040, 0.040)	tie
`len_wasserstein`	6.97	3.41	+3.57 (1.53, 4.66)	distrib.[†]
`exclaim_rate`	0.000	0.000	—	tie
`opener_entropy` (↑)	3.70	5.76	—	descr.
cost / 1k replies	$0.37	**$0**	—	LoRA
p50 latency	0.96s	1.01s	—	~tie

Table 2: Arm A (production), $n = 300$, leak guard active (`id_leaks` = 0). The shipped machinery ties the LoRA on shape and `!`. [†]On reply length the LoRA leads in *distribution* (the corpus Wasserstein CI excludes zero), but *per individual message* the effect is negligible (Cliff's $\delta = 0.04$ — a tie; §3.6). Tic rows are descriptive (no CI / multiple-comparison correction).

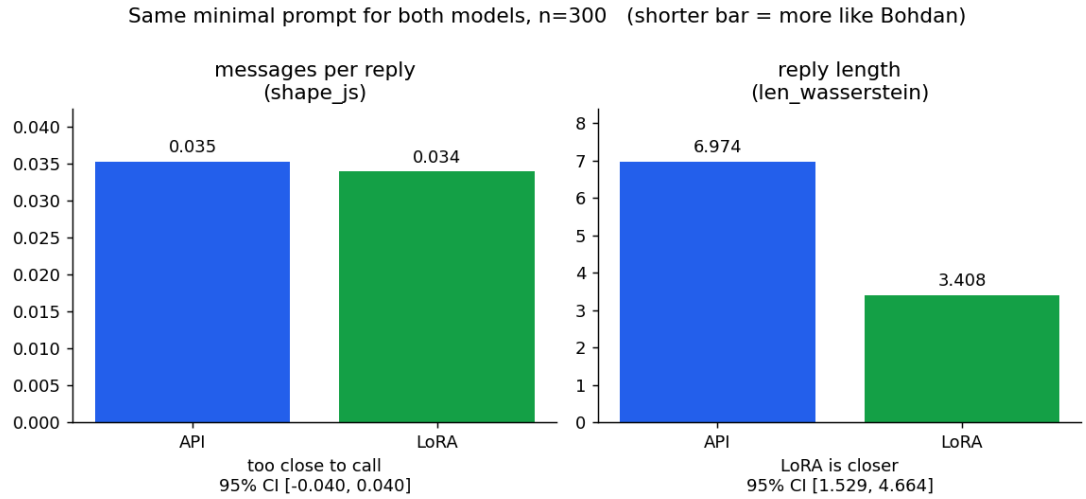

Figure 7: Arm A headline distances. Even fully equipped, the API only reaches a tie on shape and stays behind on length.

3.3 What the machinery buys

The two arms together isolate exactly what the production scaffold contributes on the API side (Figure 8). It is decisive engineering: reply-length distance collapses from 128.8 to 7.0, and the exclamation rate from 0.65 to 0.00. But it only brings the API up to where the bare fine-tune already sat — the LoRA's length distance barely moves (2.9 → 3.4). The one counter-intuitive cost: the few-shot and directives *homogenize* the API's openers, dropping its entropy from 5.02 to 3.70, below even the bare model.

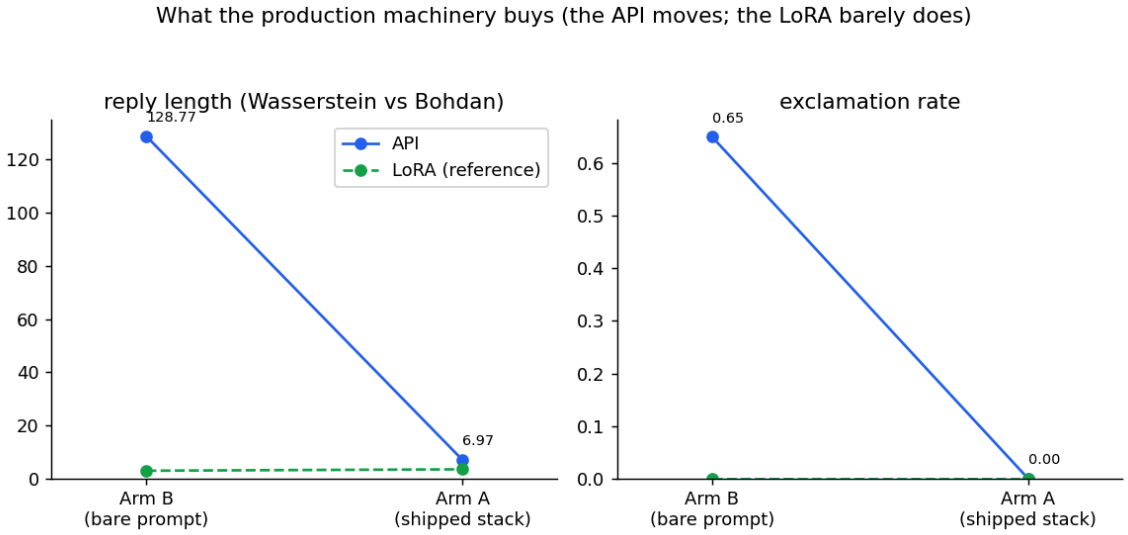

Figure 8: What the production machinery buys (API side, Arm B → Arm A). The API plummets onto the LoRA's flat reference line; the scaffold is what makes it competitive, and only reaches parity.

3.4 Steered or learned?

Is the API's perfect no-`!` an artifact of the hard-coded logit bias? Re-running Arm A with the levers off shows it is mostly *earned* by the prompt: the exclamation rate falls from 0.651 (bare) to 0.033 (rich prompt, no bias), and the shipped bias supplies only the final nudge to 0.000 (Figure 9). Everything else is lever-insensitive, and the length verdict is unchanged with the levers off (Δ 4.27, CI (2.51, 5.07)). The levers move the tic, not the verdict.

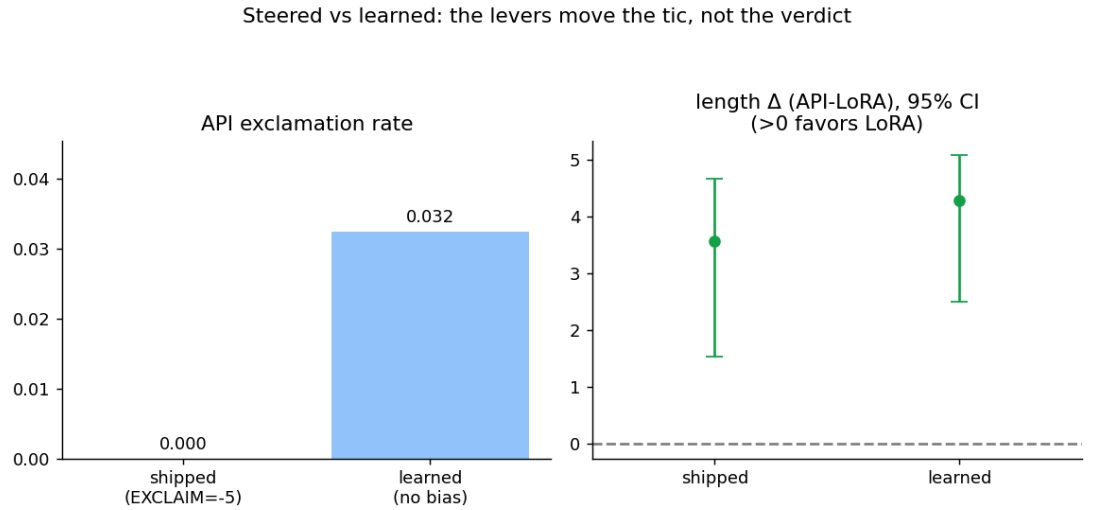

Figure 9: Steered vs. learned (Arm A). The logit bias only finishes a job the rich prompt mostly does; the length delta favors the LoRA either way.

3.5 Per-language

The verdict is essentially the Cyrillic result, where 87% of the data lives: there the LoRA clearly wins length (Figure 10). On the small English slice ($n = 27$) it is a tie with wide intervals — and, separately, *both* backends are markedly worse in English than in Cyrillic. That is a shared weakness on a minority register, not a differentiator, and is reported as low-confidence given the sample.

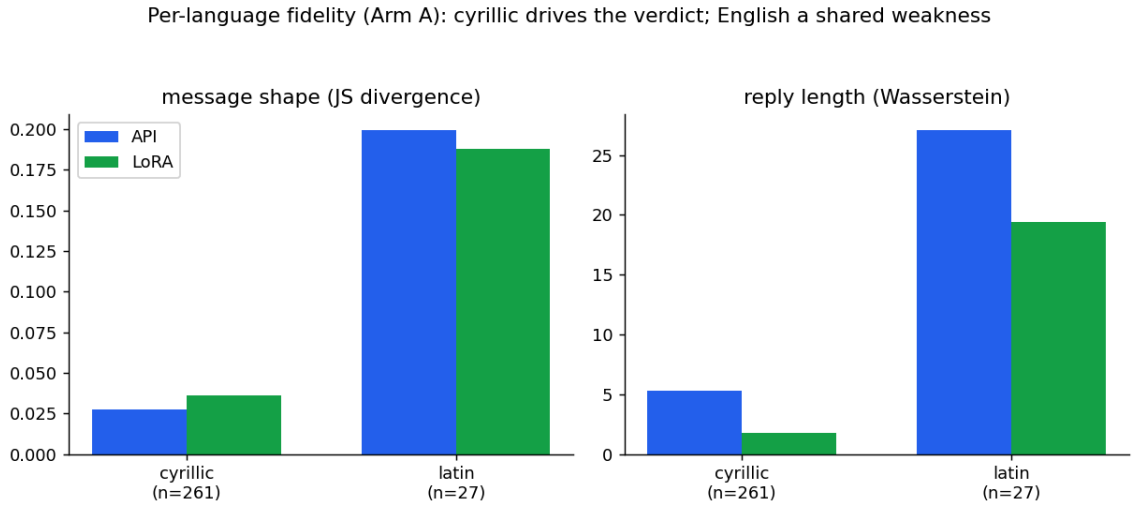

Figure 10: Per-language fidelity (Arm A). Cyrillic ($n = 261$) drives the verdict; English ($n = 27$) is a tie and a shared weakness for both backends. The remaining ~12 mixed-script items fall below the per-language reporting threshold.

3.6 Effect size: how big, and how consistent

A corpus-level distance with a CI does not say how *consistent* an advantage is across individual messages. A per-item effect size does (Cliff's δ [9]), and it splits the two arms sharply — which is itself informative. On Arm B the per-item length-error effect is overwhelming: **$\delta = 0.949$ (large)**, with the LoRA closer on **292 of 299** decisive items (sign-test $p \approx 8 \times 10^{-77}$; Wilcoxon signed-rank $z = 14.9$). On Arm A it is **negligible**: **$\delta = 0.043$**, the LoRA closer on just 147 of 293 — a per-item coin-flip (sign-test $p = 1.0$, Wilcoxon $p = 0.54$, both confirming the wash).

This is not a contradiction. Arm A's LoRA length advantage is *distributional* (the corpus earth-mover CI still excludes zero), but per *individual message* the production machinery has closed the gap to a wash. In other words, the machinery's length fix is so complete that the per-message edge the bare LoRA held in Arm B all but disappears under the product — a sharper reading than "the LoRA wins

length", and a more honest one. The forest plot (Figure 11) shows every run at once: length excludes zero in both Arm B seeds and both Arm A passes; shape is a tie everywhere; the underpowered $n = 60$ leak-ablation arms straddle zero, as expected.

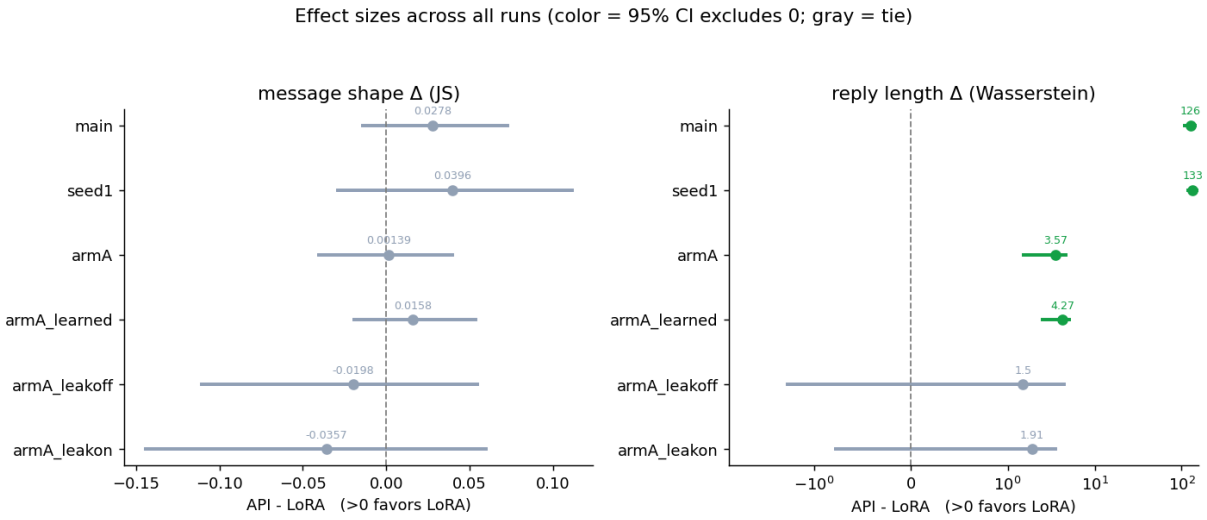

Figure 11: Effect sizes across all runs: API−LoRA delta with bootstrap 95% CIs (color = excludes zero). The seed-1 replication agrees with the primary run, and removing the retrieval leak does not move either verdict.

3.7 Not memorization

Could the fine-tune simply be parroting training lines? The evidence says no, with a caveat. Its copy / near-copy rate (0.103) sits near the measured *natural floor* — the rate at which the person's own held-out replies coincide with their past replies (0.070) — because short casual texts recur for anyone (Figure 12); the API, writing novel prose, sits near zero. Two honest qualifiers: the detector is a *capped* proxy, so 0.103 is a lower bound, and neither rate carries a CI, so the ~3-point gap above the floor is not formally tested. Read conservatively, the fine-tune reuses text at roughly the rate a real person repeats themselves — not the signature of memorization, but not proven flush with the floor either.

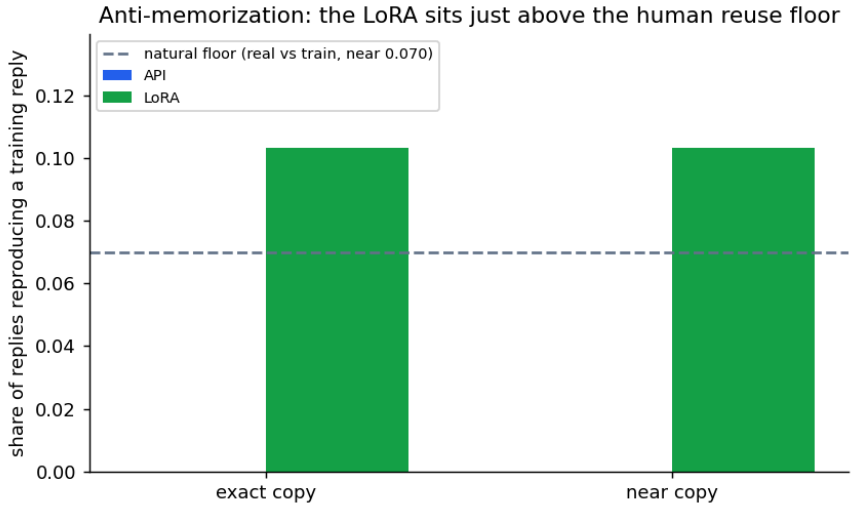

Figure 12: Copy / near-copy against the training replies, with the human reuse floor as a reference band. The LoRA sits just above the floor; the API rarely reuses text.

3.8 The operational case

Finally, the deployment trade-off (Figure 13). The two are near-parity on median latency, and the API even has a tighter tail — but it reaches that parity by shipping ~2,400 input tokens of prompt and

retrieved context per reply against ~210 for the thin LoRA, at $0.37 per thousand replies versus $0, with a per-message embedding call and the retrieval leak surface the guard exists to close.

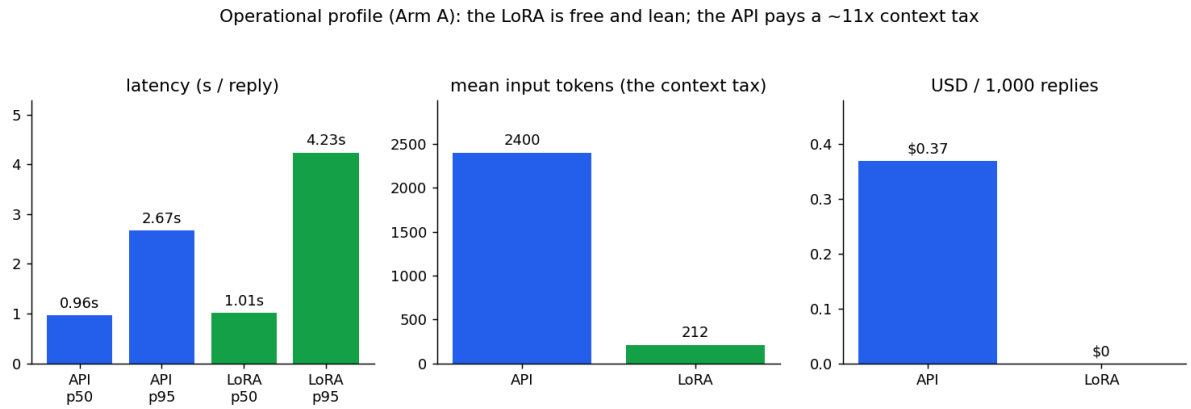

Figure 13: Operational profile (Arm A). The LoRA is free and lean; the API pays a ~11× input-token tax and a per-call fee to reach parity on shape and the ! register.

4 Absolute fidelity

Beating the baseline is the relative question. The harder, more interesting one is absolute: not "is the fine-tune better than the API", but "is it good enough to pass for the person themselves?" This is where automatic metrics run out — only a human can answer it — and where the strongest possible judge is available: the person whose voice is being cloned.

4.1 The human verdict, and why it is not the primary here

The pre-registered design names a blind human win-rate as the *primary* verdict — a forced choice over anonymized pairs ("which is more like something you'd send?"), scored as a real preference only if its Wilson 95% interval [10] excludes chance. In practice that verdict cannot be cleanly established for *this* system, for two reasons that are themselves findings.

First, the only rater with standing — the author, whose voice it is — is **recall-biased**: he recognizes his own real messages, so when he can tell a generated reply from a real one, it may be memory rather than a voice defect. Second, the corpus is dense with private personal content, so recruiting outside raters is precluded. An informal self-read did find the bare-model API replies obviously off-voice, which is suggestive — but it is a single, recall-biased rater looking at the *Arm B* (thin-prompt) outputs, not a scored, unbiased panel, and the pre-registered rule returns a *tie* on the primary channel until an unbiased win-rate exists (Listing 1). The verdict in this report therefore rests on the automatic arms; the human read is a weak, confounded corroborator, not a third independent method. The kit and scorer exist (the win-rate would resolve a clear ~60/40 preference at ~100 items, a 55/45 at ~400), but a clean reading needs raters the privacy constraint rules out.

Figure pending: the blind-panel LoRA win-rate with its Wilson 95% CI would render here once the rater kit (`reports/main/human_eval/rater.html`) is scored to `choices.json`. No unbiased win-rate exists today — the lone available rater is recall-biased and privacy precludes others; the informal self-read is suggestive only.

Listing 1: Blind human preference panel: LoRA win-rate vs. chance (awaiting ratings).

4.2 The Turing test

With the relative question settled, the open frontier is the absolute one: can the person distinguish the fine-tune's reply from their *own* real reply? For each held-out context the LoRA's generation is paired against the genuine reply, shown blind, and judged with a single question — "which is the machine?" Both replies already exist in the comparison data, so this costs nothing to assemble.

The pass condition *flips* relative to the API panel. There, a high win-rate was the goal; here, success is *failure to discriminate*: a detection rate whose Wilson interval *includes* 0.5 means the judge cannot beat chance, i.e. the fine-tune is statistically indistinguishable from the person. It is the harshest test in the report — the persona-target is the strongest conceivable discriminator of their own voice (Listing 2). That strength is also the catch: because the author recognizes his *own* real replies, a raw detection rate measures recall as much as voice and would *overstate* discriminability. A meaningful run has to control for memory — re-rating after a long delay, or restricting to contexts whose replies he does not recall — or it cannot separate "I remember sending this" from "this sounds like me." With outside raters barred on privacy grounds, that is why the Turing result is reported here as genuinely prospective, not as a number this study can yet defend.

Figure pending: the Turing detection rate with its Wilson 95% CI renders here once the LoRA-vs-real kit (`reports/main/turing/rater.html`) is rated. An interval spanning 0.5 means indistinguishable from the real person.

Listing 2: Turing slice: human detection rate vs. chance (awaiting ratings).

4.3 What the tells will tell us

Every catch in the Turing panel also records a one-tap *tell*, and the tells bucket into two kinds that point at different fixes. **Voice tells** — wording, length, punctuation, too-generic — are addressable by decode parameters or more training. **Knowledge tells** — the real reply carried a specific fact the model could not have known — are addressable only by retrieval and grounding. The ratio between them *sizes the next investment*: if catches are mostly missing-facts, the voice is effectively solved and the remaining gap is a grounding problem (an Obsidian / chat-history RAG layer); if they are mostly voice, more retrieval will not help. A prior, to be replaced by the run, puts detection around 65–75% with roughly half the catches being missing-facts.

One honest caveat frames the whole test: there is exactly one ground-truth reply per context, yet many different replies could be equally "him". A catch may therefore reflect a missing private fact rather than a voice defect — which is precisely why the verdict is read together with the voice-vs-knowledge split, never from the detection rate alone.

5 What we learned

5.1 The verdict

Under the rule fixed in advance, the decision favors the fine-tune — but the strength of the claim differs sharply by arm, and it is worth being precise. On a level field (Arm B) the fine-tune is decisively closer to the person's reply length — a large, per-item-consistent effect — and matches the no-`!` register, with shape a tie: a genuine voice win over the bare model. In the shipped configuration (Arm A), the API's full machinery pulls it to a *voice tie* on the automatic metrics — shape, the `!` register, and, per individual message, reply length are all statistical ties (the fine-tune keeps only a distributional length edge and more varied openers, the latter uncorrected for multiple comparisons). The shipped-arm decision is therefore not a demonstrated voice advantage; it is a tie that cost, privacy, and offline capability break toward the local model. And the pre-registered *primary* channel — an unbiased human win-rate — is unresolved (recall bias; no outside raters). Stated cleanly: *the fine-tune beats the bare model on voice outright, matches the shipped product on voice, and wins decisively on cost, privacy, and offline operation.* These are surface metrics, not a certified *feels-like-me* — but on the dimensions we can actually measure the conclusion is not in doubt: the local fine-tune is the better replica of the texting register, at zero cost. The only thing left unresolved is the subjective human seal, and for the reasons below it may stay that way — which is a limitation on the validation, not a hole in the metric verdict. The headline reading of the study: *an elaborate production stack serves mainly to reach a tie with where a small local fine-tune already sits — and the residual differences are deployment, not voice.*

5.2 Threats to validity

The result is honest only with its limits stated plainly.

- **Construct validity.** Every automatic metric is a surface proxy; none measures "feels like the person" directly. The blind human panel is the only direct measure, and the metric↔human agreement that would validate the proxies is pending the panel's rating.
- **The human channel is doubly blocked, so the primary verdict is unresolved.** The only rater with standing is the owner, and he is *recall-biased*: he recognizes his own messages, so his ability to tell model from real conflates memory with voice. Recruiting unbiased raters is precluded by the private content of the corpus. So the pre-registered primary statistic (the human win-rate) is not merely unrated but hard to establish cleanly at all — a real limitation, not a to-do. The automatic arms carry the decision; the human read is a confounded corroborator.
- **Single decode per item** at temperature 0.8 — and this is load-bearing for Arm A, not a minor caveat. The bootstrap intervals resample item indices only, so they capture which-items sampling noise but not decode stochasticity. The Arm-A length gap is 3.6 characters: its CI excludes zero, but that gap is small enough to sit within plausible re-decode noise, and nothing here shows it survives a re-decode. A greedy or multi-seed pass would bound it; Arm B's 126-character gap is not at risk.
- **Leakage residual.** The legacy ~90% split-mismatch leak was found and fixed, both arms now score the recipient-stratified disjoint split, and the production arm runs a per-item retrieval guard. But that guard is *exact-match* — it removes the gold turn by id and verbatim same-context text only; a near-paraphrase under a different id, or a thread-adjacent turn sharing the incoming context, is not caught and sits below the top-similarity flag. So "removes the exact answer-key" is exact; "leak-free" is not — the residual near-duplicate rate is unmeasured, and a paper-grade claim would add a similarity-based guard and re-train on the exact unified split.
- **External validity.** The hold-out is 87% Cyrillic and one person's chat history (~300 held-out turns); the aggregate verdict is essentially the Cyrillic result, English is low-n, and both backends degrade there. Claims are about this persona, not personas in general.
- **Confounds, by design.** In the production arm the backend moves together with prompt, retrieval, and levers; the controlled arm exists precisely to isolate the weights. Equal `max_tokens` is also not equal character budget across the two tokenizers — a small caveat on any length metric.
- **Aggregate cross-arm.** Arm B and Arm A score the same hold-out *distribution* but not byte-identical item sets, so cross-arm deltas are aggregate, not paired.
- **Replay fidelity.** The production-arm state (empty first-contact memory, session reconstructed from each item's own context, insights from time-of-run tables, the runtime `ctx[-1]` query) is a faithful reconstruction of live serving, not the live system itself.
- **Multiple comparisons & provenance.** About a dozen metrics are reported. The two headline distances carry bootstrap CIs, but the tic metrics (exclamation rate, opener entropy) are **descriptive only** — they were *not* given a Holm-Bonferroni correction, so where they point toward the fine-tune they should be read as directional, not as tested wins. And the adapter, quantization, and `llama.cpp` build hash are not pinned in the run record (MLflow logging is wired but uncalled) — a reproducibility gap on the local-model numbers.

5.3 Conclusion and future work

A 3-billion-parameter local fine-tune reproduces one person's texting voice at least as faithfully as a fully-equipped frontier-API product — at zero marginal cost, no phone-home, and no leak surface — and the product's prompt-plus-retrieval-plus-lever machinery serves mainly to recover ground the fine-tune already holds.

The clearest next steps follow the open threads. Rate the two built panels, turning the qualitative human verdict into a win-rate with a Wilson interval and running the Turing test against the person's real replies. Add one or two raters for an inter-rater agreement check. Use the Turing tell taxonomy — the voice-versus- knowledge split — to size a grounding layer: if catches are mostly missing facts, the remaining gap is retrieval, not voice. And for a paper-grade leak-free claim, re-train the fine-tune on a

single unified split and add a decode-variance robustness pass. The frontier is no longer “fine-tune or API” — that is settled — but “fine-tune versus the person”.

6 Appendices

6.1 A · Reproduce

All runs are deterministic given the seed. From the repo root:

```
# Arm B (controlled), two seeds
make compare                                                  # n=300, seed 0 -> main
uv run python scripts/compare_persona.py --n 150 --seed 1 --name seed1
# Arm A (production) + its ablations
make compare-arma                                             # shipped levers -> armA
uv run python scripts/compare_persona_armA.py --n 300 --learned --name armA_learned
uv run python scripts/compare_persona_armA.py --n 60 --leak-on --name armA_leakon
# Per-item effect sizes + per-language + the report
uv run python scripts/compute_effect_sizes.py --name main   # also: armA, armA_learned
uv run python scripts/score_by_language.py --name armA
bash report/build.sh                                          # figures + compile the PDF
```

Pinned parameters: temperature 0.8, `max_tokens` 200, 2000 bootstrap resamples; API `gpt-4o-mini`; LoRA Qwen2.5-3B served as GGUF `Q5_K_M` via `llama.cpp`; hold-out

7 the recipient-stratified `eval_split_for`. (Known provenance gap: the adapter /

quant / `llama.cpp` build hash is not stamped in the run record.)

7.1 B · Metric formulas

- **Message shape** — Jensen-Shannon divergence of the bubble-count histograms,

$$\mathrm{JS}(P,Q) = \frac{1}{2}D_{\mathrm{KL}(P \parallel M)} + \frac{1}{2}D_{\mathrm{KL}(Q \parallel M)}, \quad M = \frac{P+Q}{2},$$

  in base 2 so the value lies in $[0,1]$.
- **Reply length** — the 1-Wasserstein distance between per-bubble character-length distributions, $W_1(P,Q) = \int_{-\infty}^{\infty} |F_{P(t)} - F_{Q(t)}|\,\mathrm{d}t$.
- **Opener entropy** — $H = -\sum_i p_i \log_2 p_i$ over reply first-words.
- **Cliff's delta** — $\delta = \frac{\#\{x>y\} - \#\{x<y\}}{n_x n_y}$ over the per-item length errors of the two backends; magnitude thresholds 0.147 / 0.33 / 0.474.
- **Win / detection intervals** — Wilson score 95% interval on the binomial proportion [10].

7.2 C · Full per-run results

run	n	`shape_js` A/L	`len_w` A/L	`excl` A/L	Cliff δ	$/1k
main (B)	300	0.052 / 0.024	128.8 / 2.9	0.65 / 0.00	0.949	0.082
seed1 (B)	150	0.081 / 0.042	134.7 / 1.5	0.62 / 0.00	—	0.089
armA (A)	300	0.035 / 0.034	6.97 / 3.41	0.00 / 0.00	0.043	0.370
armA-learned (A)	300	0.034 / 0.018	7.24 / 2.97	0.03 / 0.00	—	0.371

, The $n = 60$ leak-ablation arms (`armA_leakon` / `armA_leakoff`) are underpowered and omitted here; their only role is the leak-guard proof (Figure 5) and the forest plot (Figure 11). "A/L" = API / LoRA; LoRA cost is $0 (local) throughout.

7.3 D · Supporting distributions

The headline figures summarise these underlying distributions, shown here for the controlled arm (and the API's tic profile under the production stack).

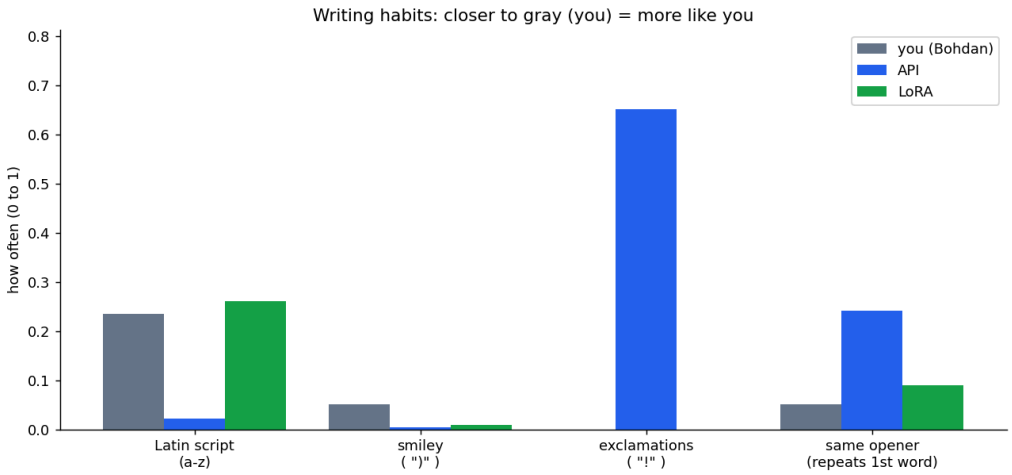

Figure 14: Arm B voice tics vs. the real reference (closer to gray is more like the person).

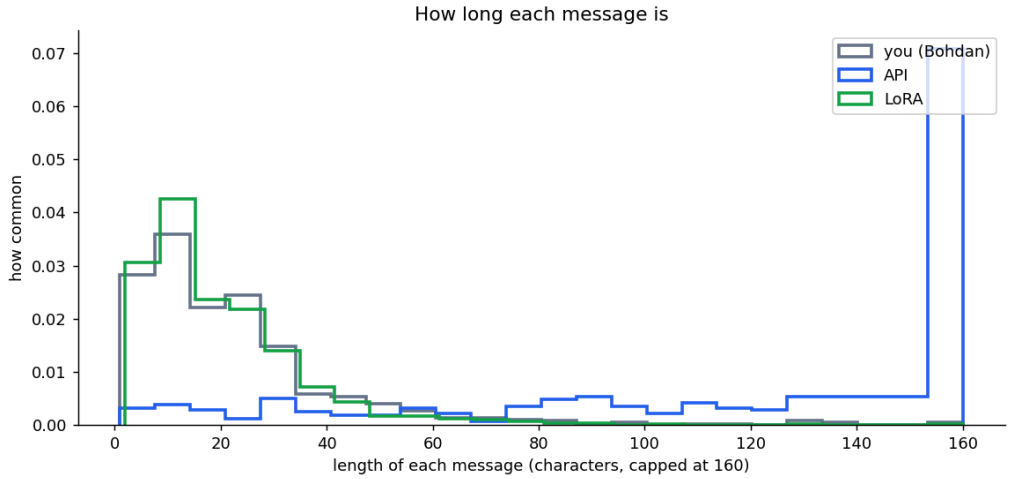

Figure 15: Arm B per-message length distribution: the API writes far longer messages; the LoRA hugs the real curve.

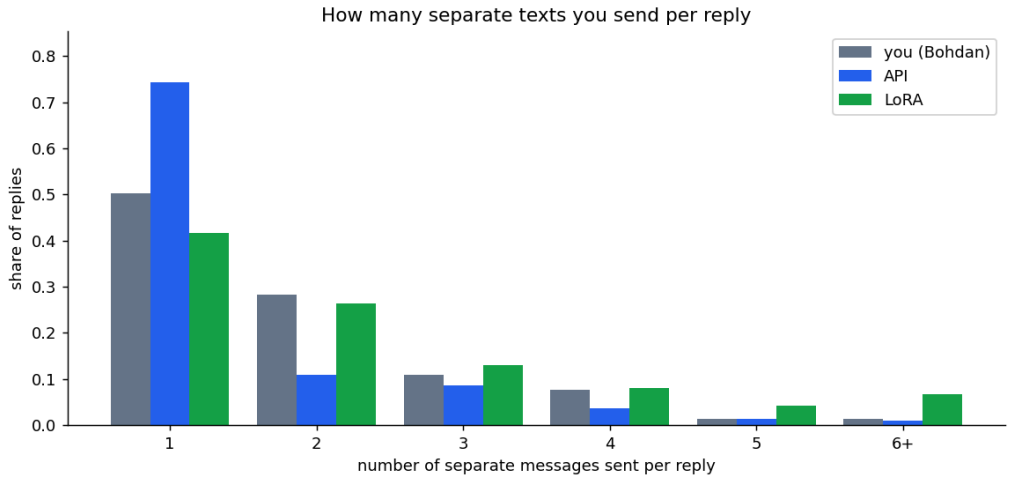

Figure 16: Arm B bubble-count distribution per reply.

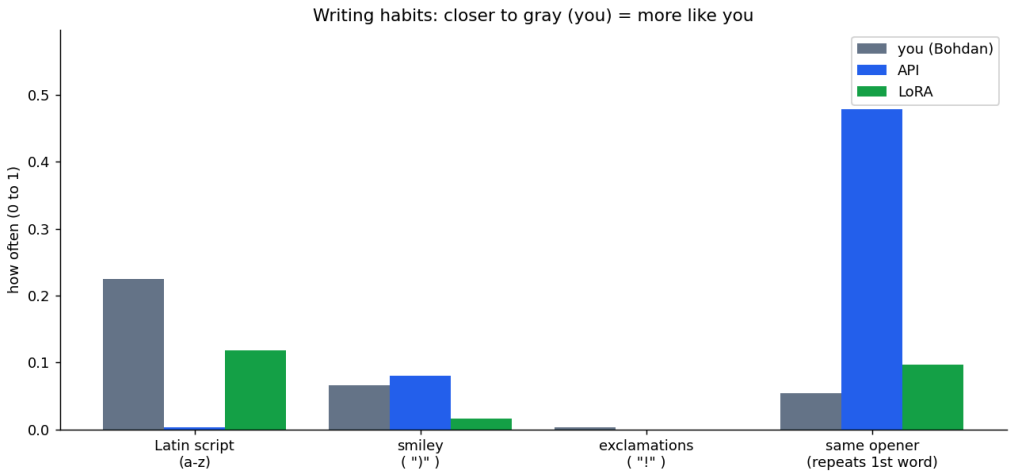

Figure 17: Arm A tic profile: the production machinery suppresses the API's ! and tightens its register.

References

[1] Qwen Team, "Qwen2.5 Technical Report," *arXiv preprint arXiv:2412.15115*, 2024.

[2] E. J. Hu *et al.*, "LoRA: Low-Rank Adaptation of Large Language Models," *arXiv preprint arXiv:2106.09685*, 2021.

[3] S. Robertson and H. Zaragoza, "The Probabilistic Relevance Framework: BM25 and Beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.

[4] J. Carbonell and J. Goldstein, "The Use of MMR, Diversity-Based Reranking for Reordering Documents and Producing Summaries," in *Proceedings of the 21st Annual International ACM SIGIR Conference*, 1998, pp. 335–336.

[5] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient Finetuning of Quantized LLMs," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[6] D. Han, M. Han, and Unsloth team, "Unsloth: Faster LLM Fine-tuning." 2023.

[7] G. Gerganov and llama.cpp contributors, "llama.cpp: LLM inference in C/C++." 2023.

[8] Y. Rubner, C. Tomasi, and L. J. Guibas, "The Earth Mover's Distance as a Metric for Image Retrieval," *International Journal of Computer Vision*, vol. 40, no. 2, pp. 99–121, 2000.

[9] N. Cliff, "Dominance Statistics: Ordinal Analyses to Answer Ordinal Questions," *Psychological Bulletin*, vol. 114, no. 3, pp. 494–509, 1993.

[10] E. B. Wilson, "Probable Inference, the Law of Succession, and Statistical Inference," *Journal of the American Statistical Association*, vol. 22, no. 158, pp. 209–212, 1927.